# Keylogger Modification Documentation

## 1. Purpose and Scope

This document describes the changes shown in the supplied modified version of mainfile.py, compared with the original Python-Keylogger/keylogger.py. It also documents the supplied aes_encrypt_decrypt.py module and explains how the two modules work together.
The existing Python source files were not modified while creating this document. Note that the workspace copy of mainfile.py was empty when inspected; the detailed mainfile.py implementation described below comes from the attached source content.
This functionality should only be used on devices and accounts for which the operator has explicit authorization. Keyboard monitoring can capture passwords, messages, and other sensitive information.

## 2. High-Level Change Summary

The original script was a basic keyboard-event logger that:

- Listened for key-press events using pynput.
- Converted each key event to text.
- Performed limited translation for Space, Shift, Ctrl, and Enter.
- Appended the result to log.txt.

Continued running until externally stopped.
The modified implementation adds:

- A named log-file configuration.
- A named encrypted-output configuration.
- An encryption-password configuration.
- AES-GCM encryption when the logger is stopped.
- An explicit ESC-key stop action.
- Additional handling for Tab, Backspace, and Escape.
- Safer file operations that suppress OSError exceptions.
- A __main__ guard so importing the module does not automatically start the listener.
- Startup and shutdown status messages.
- A separate AES-256-GCM encryption/decryption utility that supports password-derived keys and optional raw 256-bit keys

## 3. Step-by-Step Modification History

This section presents the changes in the order they were introduced conceptually, starting with the original basic logger and ending with the encrypted-log workflow.

### Step 1: Rename the main script and establish a larger application structure

The original file was named keylogger.py. The modified implementation was organized as mainfile.py and was designed to work with a separate module named aes_encrypt_decrypt.py.
The purpose of this separation is:

- mainfile.py handles keyboard events and decides when logging starts and stops.
- aes_encrypt_decrypt.py handles encryption and decryption of files.

The logger calls the encryption module instead of implementing cryptographic operations inside the keyboard callback.
This separates event handling from cryptographic file processing and makes the encryption functions reusable from both Python code and the command line.

## Step 2: Expand the imports

The original import was:

```python
from pynput.keyboard import Listener
```

The modified version changes it to:

```python
import os
from contextlib import suppress
from pynput.keyboard import Key, Listener
from aes_encrypt_decrypt import encrypt_file
```

The changes were made for these reasons:

- os provides os.path.exists, which checks whether the plaintext log exists before encryption.
- suppress allows selected OSError exceptions to be ignored while opening or writing the log.
- Key provides named constants such as Key.esc, Key.tab, and Key.backspace.
- Listener is retained because it is still required to receive keyboard events.
- encrypt_file imports the encryption operation from the separate AES module.

## Step 3: Move filenames and the password into configuration constants

The original implementation embedded the filename directly in the write operation:

```python
open("log.txt", "a")
```

The modified implementation introduces:

```python
LOG_FILE = "vvlogs.txt"
ENCRYPTED_LOG = "vvlogs.txt.enc"
```

```
PASSWORD = "Varma@8135"
```

The resulting changes are:

- The plaintext log has a named configuration value, LOG_FILE.
- The encrypted output has a separate name, ENCRYPTED_LOG.
- The encryption password is available to the shutdown routine through PASSWORD.

Filenames are no longer repeated in multiple parts of the program.
This also introduces a security concern: the password is stored in source code. Anyone who can read the source can recover the password and decrypt the output.

## Step 4: Preserve the original character conversion

The original conversion logic was retained:

Python
```
letter = str(key)
letter = letter.replace("'", "")
```

The process is unchanged:

- Convert the pynput key object to text.
- Remove single-quote characters added around ordinary character values.

Store the resulting text in letter before applying special-key rules.
For example, a character represented as 'a' becomes a.

## Step 5: Convert the original independent conditions into an if/elif chain

The original code checked each supported key with a separate if statement. The modified code uses one mutually exclusive chain:

Python
```
if letter == "Key.space":
    letter = " "
elif letter == "Key.shift_r" or letter == "Key.ctrl_l":
    letter = ""
elif letter == "Key.enter":
    letter = "\n"
```

The behavior remains:

- Space is converted to a normal space.
- Right Shift and left Ctrl are discarded.

Enter is converted to a newline.
Using elif makes it clear that one received key should be processed by one matching rule.

## Step 6: Add Tab, Backspace, and Escape handling

Three new rules were added:

```python
elif letter == "Key.tab":
    letter = "\t"
elif letter == "Key.backspace":
    letter = "[BACKSPACE]"
elif letter == "Key.esc":
    letter = "[ESC]"
```

The effect of each addition is:

- Tab is stored as a real tab character instead of the literal text Key.tab.
- Backspace is stored as [BACKSPACE], which records the key event without attempting to edit already-written log text.

Escape is stored as [ESC] when its press callback runs; its release is separately used as the stop signal.
Any key not listed in the conditions continues to use its string representation.

## Step 7: Change the log destination and encoding

The original write operation was:

```python
with open("log.txt", 'a') as f:
    f.write(letter)
```

The modified write operation is:

```python
with suppress(OSError), open(LOG_FILE, "a", encoding="utf-8") as
f:
    f.write(letter)
```

The changes are:

- log.txt is replaced by the configurable LOG_FILE value.
- Append mode is retained, so new events are added to the existing plaintext log.
- UTF-8 is explicitly selected for predictable text encoding.

The file operation is placed inside suppress(OSError).
The exception suppression prevents an operating-system file error from immediately stopping the callback. However, it also hides the reason for a failed write, which can make lost log data difficult to diagnose.

## Step 8: Add an on_release callback

The original script had only write_to_file, which handled key presses. The modified script adds a second callback:

```python
def on_release(key):
```

This callback is needed because the program must detect a key-release event for ESC and use it as the controlled shutdown event.

## Step 9: Use ESC as the controlled stop key

Inside on_release, the modified version checks:

```python
if key == Key.esc:
```

When ESC is released, the following sequence occurs:

- The program opens LOG_FILE in append mode.
- It writes a newline and the marker [KEYLOGGER STOPPED].
- It checks whether LOG_FILE exists with os.path.exists.
- If the file exists, it calls encrypt_file.

It returns False.
Returning False tells pynput to stop the listener. This replaces the original behavior, where the listener continued until the process was externally interrupted.

## Step 10: Encrypt the log during shutdown

The modified shutdown code calls:

```python
encrypt_file(LOG_FILE, ENCRYPTED_LOG, password=PASSWORD)
```

The arguments mean:

- Read the plaintext log from LOG_FILE.
- Write the encrypted result to ENCRYPTED_LOG.

Use the configured password to derive the AES key.
The encryption operation occurs after the stop marker is written, so the encrypted file includes the final shutdown marker. The source code does not delete the plaintext file after encryption, so both files may remain on disk.

## Step 11: Register both listener callbacks

The original listener was:

```python
with Listener(on_press=write_to_file) as l:
    l.join()
```

The modified listener is:

```python
with Listener(on_press=write_to_file, on_release=on_release) as listener:
    listener.join()
```

The changes are:

- on_press=write_to_file continues to process pressed keys.
- on_release=on_release detects the release of ESC.
- The listener variable is renamed from l to listener, making its purpose clearer.
- listener.join() keeps the program active until the callback returns False or the process is otherwise stopped.

## Step 12: Add a __main__ guard

The original listener started as soon as the module was executed or imported. The modified implementation places startup under:

```python
if __name__ == "__main__":
```

This changes module behavior in two ways:

- Running python mainfile.py starts the listener.

Importing mainfile.py from another module defines its functions and constants without automatically starting keyboard monitoring.
This is a standard Python structure that makes the module easier to reuse and test.

## Step 13: Add startup messages

Before creating the listener, the modified version prints:

```python
print(f"Keylogger started. Press ESC to stop and encrypt {ENCRYPTED_LOG}.")
print(f"Password: {PASSWORD}")
```

The first message explains the current state and stop action. The second displays the configured password. Displaying the password is unsafe because it exposes the credential in the terminal history or console output; it should not be used in a secure implementation.
Step 14: Create the AES encryption module
The separate aes_encrypt_decrypt.py module adds file encryption and decryption independently of the keyboard listener. It defines:

- Constants for the file format and cryptographic sizes.
- A password-to-key derivation function.
- A raw hexadecimal key parser.
- An encrypt_file function.
- A decrypt_file function.

A command-line interface with encrypt and decrypt subcommands.
This module is the new dependency used by mainfile.py at shutdown.

## Step 15: Define AES-GCM and PBKDF2 settings

The encryption module configures:

```python
KEY_SIZE = 32
SALT_SIZE = 16
NONCE_SIZE = 12
KDF_ITERATIONS = 390_000
```

These values mean:

- KEY_SIZE = 32 creates a 256-bit AES key.
- SALT_SIZE = 16 creates a 16-byte password salt.
- NONCE_SIZE = 12 creates a 12-byte AES-GCM nonce.

KDF_ITERATIONS = 390_000 controls the PBKDF2 work factor.
The module uses SHA-256 with PBKDF2 to derive an AES key from a password, then uses AES-GCM for authenticated encryption.

## Step 16: Generate a password-derived AES key

The derive_key_from_password function receives a password and salt. It creates a PBKDF2-HMAC-SHA256 object and derives a 32-byte key.
During encryption, the salt is randomly generated and stored in the encrypted file. During decryption, the stored salt is read and combined with the same password to recreate the same AES key.
The password itself is not stored in the encrypted file. In the current project, however, it is present as a hard-coded default in the source code.

## Step 17: Support optional raw 256-bit keys

The parse_raw_key_hex function accepts a hexadecimal string, decodes it, and verifies that it contains exactly 32 bytes. This allows the encryption module to use a raw AES-256 key instead of a password-derived key.
The two modes are selected as follows:

- If raw_key_hex is supplied, raw-key mode is used.

Otherwise, password mode is used.
A raw-key encrypted file must be decrypted with the same raw key.
A password-encrypted file must be decrypted with the same password.

## Step 18: Encrypt the plaintext log

The encrypt_file function performs these operations:

- Select password mode or raw-key mode.
- Derive or parse the 32-byte AES key.
- Generate a random 12-byte nonce.
- Read the plaintext input file as bytes.
- Encrypt the bytes with AES-GCM.

Write the file header and encrypted content.

In password mode, the output contains the magic value, mode byte, salt, nonce, and ciphertext with the GCM authentication tag. In raw-key mode, the salt is omitted because no password derivation is performed.

## Step 19: Decrypt and authenticate the encrypted file

The decrypt_file function reverses the encryption process:

- Read and validate the AESGCMv1 magic value.
- Read the mode byte.
- Read the salt when the file uses password mode.
- Read the nonce and ciphertext.
- Derive or parse the correct AES key.
- Authenticate and decrypt the ciphertext with AES-GCM.

Write the recovered plaintext to the requested output path.
If the password or key is incorrect, or if the encrypted content has been changed, AES-GCM authentication fails and the module raises a decryption error.

## Step 20: Add the encryption module command-line interface

The module defines argparse subcommands:

```shell
encrypt -i INPUT -o OUTPUT
decrypt -i INPUT -o OUTPUT
```

Both commands also accept an optional password or raw key. This means the encryption module can be used in two ways:

- Called from mainfile.py when the listener stops.
- Run independently from a terminal to encrypt or decrypt other files.

Step 21: Complete the final runtime workflow
After all changes, the complete workflow is:

1. Run mainfile.py directly.
2. The __main__ guard permits startup.
3. The program prints its startup status.
4. The listener receives key presses and calls write_to_file.
5. write_to_file converts and normalizes each key.
6. The normalized value is appended to the plaintext log.
7. When ESC is released, on_release writes the stopped marker.
8. encrypt_file reads the plaintext log.
9. PBKDF2 derives an AES-256 key from the configured password.
10. AES-GCM encrypts and authenticates the log.
11. The encrypted output is written to ENCRYPTED_LOG.

Returning False stops the listener and exits the with block.
This is the complete progression from the original append-only logger to the supplied event logger with encrypted shutdown output.

# 4. Original Implementation: keylogger.py

## 4.1 Imports

The original script imports only Listener from pynput.keyboard:

```python
from pynput.keyboard import Listener
```

It does not import Key, operating-system helpers, exception utilities, or an encryption function.

## 4.2 Key Conversion

```python
letter = str(key)
letter = letter.replace("'", "")
```

This removes quotation marks from ordinary character representations such as 'a', allowing the logged value to become a.

## 4.3 Special-Key Handling

The original version uses independent if statements for four values:

- Key.space becomes a space character.
- Key.shift_r becomes an empty string.
- Key.ctrl_l becomes an empty string.

Key.enter becomes a newline.
No output is written for the two handled modifier keys. Other special keys are written using their string representation, for example Key.tab would be logged as Key.tab if encountered.

## 4.4 File Output

The original script always appends to a file named log.txt:

```python
with open("log.txt", "a") as f:
    f.write(letter)
```

The file is opened in append mode, and no explicit encoding is supplied. There is also no exception handling around the file operation.

### 4.5 Listener Lifetime

The listener starts immediately when the file is executed:

```python
with Listener(on_press=write_to_file) as l:
    l.join()
```

The listener registers only an on_press callback. There is no application-defined stop key, so the process must be interrupted or terminated externally.

# 5. Modified Implementation: mainfile.py

## 5.1 Expanded Imports

The modified version adds:

```python
import os
from contextlib import suppress
from pynput.keyboard import Key, Listener
from aes_encrypt_decrypt import encrypt_file
```

Each addition has a specific role:

- os is used to check whether the log file exists before encryption.
- suppress is used to ignore OSError during log-file access.
- Key provides named key constants, including Key.esc.
- encrypt_file connects the logger to the AES encryption module.

## 5.2 Centralized Configuration Values

The modified script replaces the hard-coded output filename inside write_to_file with named constants:

- LOG_FILE identifies the plaintext log file.
- ENCRYPTED_LOG identifies the encrypted output file.

PASSWORD supplies the password passed to the encryption function.
This makes the output locations and encryption configuration visible in one place and avoids repeating filenames in multiple functions. The implementation contains a hard-coded password, which is a security weakness because anyone who can read the source can recover it.

## 5.3 Revised Key Translation Logic

The conversion and quote-removal steps remain the same:

```python
letter = str(key)
letter = letter.replace("'", "")
```

The special-key handling was expanded and changed from separate if statements to an if/elif chain:

- Key.space becomes a regular space.
- Key.shift_r and Key.ctrl_l are discarded.
- Key.enter becomes a newline.
- Key.tab becomes a tab character.
- Key.backspace becomes the visible marker [BACKSPACE].

Key.esc becomes the visible marker [ESC].
Using elif means that once one key condition matches, the remaining conditions are skipped. In this particular set of mutually exclusive key values, the result is equivalent to the original independent checks while making the intent clearer.
The modified version still does not translate every possible modifier or special key. Any unhandled key is still written using its string form.

## 5.4 More Defensive Log Writing

The modified version writes to the configured log file using:

```python
with suppress(OSError), open(LOG_FILE, "a", encoding="utf-8") as
f:
    f.write(letter)
```

Changes from the original are:

- The configured LOG_FILE constant is used.
- UTF-8 encoding is explicitly selected.

OSError is suppressed, so failures such as an unavailable path or permission error do not immediately terminate the callback.
Suppressing the exception keeps the listener running, but it also hides logging failures. In a production-quality authorized monitoring tool, failures should normally be reported through a controlled diagnostic path rather than silently ignored.
5.5 New Release Callback
The modified script introduces on_release(key). It checks whether the released key is Key.esc:

- Appends a shutdown marker, [KEYLOGGER STOPPED] surrounded by newlines, to the plaintext log.
- Checks whether the plaintext log exists.
- Calls encrypt_file to create the encrypted log using the configured password.

Returns False to stop the pynput listener.
This is the main lifecycle change: pressing ESC now provides an application-defined shutdown path and triggers encryption before the listener exits.

The implementation suppresses OSError while writing the shutdown marker, but encryption errors are not suppressed. Therefore, an encryption failure can propagate out of the release callback instead of being silently ignored.

### 5.6 Main-Module Guard

The modified version moves listener startup under:

```python
if __name__ == "__main__":
```

5.7 User-Facing Status Messages
5.7 User-Facing Status Messages
Before starting the listener, the modified script prints:

- A message stating that the keylogger has started and that ESC stops it and triggers encryption.

The configured password.
The second message exposes the password on the console and should be removed or replaced with a non-sensitive status message in a secure implementation.

### 5.8 Listener Configuration

The modified listener registers both callbacks:

```python
with Listener(on_press=write_to_file, on_release=on_release) as listener:
    listener.join()
```

Compared with the original, it now receives key-press events for logging and key-release events for detecting ESC and stopping the listener.

# 6. Encryption Module: aes_encrypt_decrypt.py

### 6.1 Encryption Algorithm

The supplied module uses AES-GCM through cryptography.hazmat.primitives.ciphers.aead.AESGCM.

- AES key size: 256 bits, represented by KEY_SIZE = 32 bytes.
- GCM nonce size: 12 bytes.
- Password salt size: 16 bytes.
- Password derivation: PBKDF2-HMAC-SHA256.
- PBKDF2 work factor: 390,000 iterations.

Authentication: GCM supplies an authentication tag, allowing tampering or incorrect passwords to be detected during decryption.
AES-GCM encrypts and authenticates the plaintext. It does not hide the fact that the file exists or protect the password if the password is embedded in accessible source code.

## 6.2 Password-Derived Key Mode

When encrypt_file receives a password and no raw key, it:

- Generates a random 16-byte salt with os.urandom.
- Derives a 32-byte AES key using PBKDF2-HMAC-SHA256.
- Generates a random 12-byte nonce.
- Reads the input file as bytes.
- Encrypts the bytes with AES-GCM.

Writes the file header, mode byte, salt, nonce, and ciphertext with its authentication tag.

## 6.3 Raw-Key Mode

The module also accepts a raw key as a 64-character hexadecimal string. It validates that the decoded value is exactly 32 bytes. In this mode:

- No password-derived salt is stored.
- The file mode identifies raw-key encryption.

The same raw key must be supplied to decrypt the file.
This mode is separate from password mode and should not be mixed with it.

## 6.4 File Format

The module writes an application-specific binary format:

- The AESGCMv1 magic value.
- One mode byte.
- A 16-byte salt when password mode is selected.
- A 12-byte nonce.

AES-GCM ciphertext followed by its authentication tag.
The decryptor validates the magic value and mode, reads the fields for that mode, derives or parses the key, and then authenticates and decrypts the ciphertext.

## 6.5 Error Handling

The decryption function raises descriptive ValueError exceptions for:

- An invalid magic value.
- A missing mode byte.
- A missing salt.
- An unsupported mode.
- A missing raw key.

An incorrect password, incorrect key, or corrupted ciphertext.
The final decryption error intentionally groups wrong credentials and corruption into one message rather than revealing which condition occurred.
6.6 Command-Line Interface
The encryption module also provides an argparse command-line interface with two subcommands:

- encrypt accepts an input file, output file, and optional password or raw key.

decrypt accepts an encrypted input file, output file, and optional password or raw key.
If no password is supplied in password mode, the module falls back to its built-in default password. This is convenient for testing but insecure for real use.

# 7. End-to-End Execution Flow

## 7.1 Startup

1. mainfile.py is run directly.
2. The __main__ guard evaluates to true.
3. Startup messages are printed.
4. A pynput listener begins receiving key events.

## 7.2 During Typing

1. A key press invokes write_to_file.
2. The key is converted to a string and normalized.
3. The normalized value is appended to the plaintext log file.

## 7.3 Shutdown with ESC

1. ESC is released.
2. on_release recognizes Key.esc.
3. A stopped marker is appended to the log.
4. The plaintext log is encrypted into the configured encrypted-log file.
5. False is returned, causing the listener to stop.
6. The with block closes the listener.

# 8. Before-and-After Comparison

| Area | Original keylogger.py | Modified mainfile.py |
|---|---|---|
| Input callback | on_press only | on_press and on_release |
| Log filename | Fixed log.txt | Configured through LOG_FILE |
| Encryption | None | AES-GCM encryption on ESC |
| Stop behavior | External interruption required | ESC stops the listener |
| Key mappings | Space, right Shift, left Ctrl, Enter | Those mappings plus Tab, Backspace, and ESC |
| Encoding | Not specified | Explicit UTF-8 |
| File errors | Unhandled | OSError suppressed during log writes |
| Import behavior | Starts listener immediately | Starts only when run directly |
| Status output | None | Startup and password messages |
| Encryption utility | Not present | Password mode and raw-key mode |

# 9. Important Security Observations

- The tool records keyboard input, which may include credentials, private messages, and financial or personal data.
- The password is hard-coded in the source and printed to the console. Anyone with source or console access can obtain it.
- The plaintext log remains on disk after encryption unless it is separately deleted or securely handled.
- Encryption occurs only when ESC is released. A crash, forced termination, power loss, or system shutdown may leave recent plaintext input unencrypted.
- suppress(OSError) hides write failures, so the operator may not know that events were not recorded.
- The implementation has no access control, consent mechanism, retention policy, secure password storage, or audit trail.
- The encrypted output protects file contents only when the password or raw key remains secret.
- A safer authorized design would avoid capturing arbitrary keyboard input and would use explicit consent, secure key management, protected storage, and clear retention and deletion rules.

# 10. Dependencies and Assumptions

The implementation requires:

- Python.
- The pynput package for keyboard event monitoring.
- The cryptography package for AES-GCM and PBKDF2.
- Both Python modules to be importable from the same project or Python path.

Permission to receive keyboard events on the operating system.

The supplied modified code assumes that aes_encrypt_decrypt.py exposes an encrypt_file function with the signature used by mainfile.py.